

```

#include <stdio.h>
#include <stdlib.h>
struct Node {
    int data;
    struct Node *next;
};
struct Node *head = NULL;
void createList(int n) {
    struct Node *newNode, *temp;
    int data, i;
    if (n <= 0) {
        printf("Number of nodes should be greater than 0.\n");
        return;
    }
    for (i = 1; i <= n; i++) {
        newNode = (struct Node*)malloc(sizeof(struct Node));
        if (newNode == NULL) {
            printf("Memory allocation failed.\n");
            return;
        }
        printf("Enter data for node %d: ", i);
        scanf("%d", &data);
        newNode->data = data;
        newNode->next = NULL;
        if (head == NULL) {
            head = newNode; // first node
        } else {
            temp->next = newNode; // link new node
        }
        temp = newNode; // move temp to last node
    }
    printf("\nLinked list created successfully.\n");
}
void insertAtBeginning(int data) {
    struct Node *newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->next = head;
    head = newNode;
    printf("Node inserted at the beginning.\n");
}
void insertAtEnd(int data) {
    struct Node *newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->next = NULL;

```

```

    if (head == NULL) {
        head = newNode;
    } else {
        struct Node *temp = head;
        while (temp->next != NULL)
            temp = temp->next;
        temp->next = newNode;
    }
    printf("Node inserted at the end.\n");
}

void insertAtPosition(int data, int pos) {
    int i;
    struct Node *newNode, *temp = head;
    if (pos < 1) {
        printf("Invalid position.\n");
        return;
    }
    if (pos == 1) {
        insertAtBeginning(data);
        return;
    }
    newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    for (i = 1; i < pos - 1 && temp != NULL; i++)
        temp = temp->next;
    if (temp == NULL) {
        printf("Position out of range.\n");
        free(newNode);
    } else {
        newNode->next = temp->next;
        temp->next = newNode;
        printf("Node inserted at position %d.\n", pos);
    }
}

void displayList() {
    struct Node *temp = head;
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }
    printf("\nLinked List: ");
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
}

```

```

    }
    printf("NULL\n");
}
int main() {
    int choice, n, data, pos;

    while (1) {
        printf("\n--- Singly Linked List Operations ---\n1. Create Linked List\n2. Insert at
Beginning\n3. Insert at Any Position\n4. Insert at End\n5. Display List\n6. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
        switch (choice) {
        case 1:
            printf("Enter number of nodes: ");
            scanf("%d", &n);
            createList(n);
            break;
        case 2:
            printf("Enter data to insert: ");
            scanf("%d", &data);
            insertAtBeginning(data);
            break;
        case 3:
            printf("Enter data and position: ");
            scanf("%d %d", &data, &pos);
            insertAtPosition(data, pos);
            break;
        case 4:
            printf("Enter data to insert: ");
            scanf("%d", &data);
            insertAtEnd(data);
            break;
        case 5:
            displayList();
            break;
        case 6:
            printf("Exiting...\n");
            exit(0);
        default:
            printf("Invalid choice. Try again.\n");
        }
    }
    return 0;
}

```

--- Singly Linked List Operations ---

1. Create Linked List
2. Insert at Beginning
3. Insert at Any Position
4. Insert at End
5. Display List
6. Exit

Enter your choice: 1

Enter number of nodes: 3

Enter data for node 1: 10

Enter data for node 2: 20

Enter data for node 3: 30

Linked list created successfully.

--- Singly Linked List Operations ---

1. Create Linked List
2. Insert at Beginning
3. Insert at Any Position
4. Insert at End
5. Display List
6. Exit

Enter your choice: 2

Enter data to insert: 5

Node inserted at the beginning.

--- Singly Linked List Operations ---

1. Create Linked List
2. Insert at Beginning
3. Insert at Any Position
4. Insert at End
5. Display List
6. Exit

Enter your choice: 3

Enter data and position: 15 3

Node inserted at position 3.

--- Singly Linked List Operations ---

1. Create Linked List
2. Insert at Beginning
3. Insert at Any Position
4. Insert at End
5. Display List
6. Exit

Enter your choice: 4

Enter data to insert: 35

Node inserted at the end.

--- Singly Linked List Operations ---

1. Create Linked List
2. Insert at Beginning
3. Insert at Any Position
4. Insert at End
5. Display List
6. Exit

Enter your choice: 5

Linked List: 5 -> 10 -> 15 -> 20 -> 30 -> 35 -> NULL

--- Singly Linked List Operations ---

1. Create Linked List
2. Insert at Beginning
3. Insert at Any Position
4. Insert at End
5. Display List
6. Exit

Enter your choice: 6

Exiting...

Process returned 0 (0x0) execution time : 58.904 s

Press any key to continue.